MIESC

Multi-layer Intelligent Evaluation for Smart Contracts

Valantis STEX — Focused Triage Report

Target: ValantisLabs/valantis-stex @ 27b5c74 · Scope: src/ (37 contracts, excl. mocks/interfaces)
Scan: 246 findings (3 High / 34 Med / 57 Low / 152 Info) · Triage verdict: 0/3 High exploitable
Date: 2026-10-27 · Prepared by: MIESC automated static analysis (Slither) + manual triage

PUBLIC

Valantis STEX — MIESC audit triage memo

Target: `ValantisLabs/valantis-stex` @ `27b5c74` · scope `src/` (37 contracts, excl. `mocks/` + `interfaces/`) **What it is:** Stake Exchange AMM for liquid-staked HYPE (Hyperliquid ecosystem) — a Valantis Sovereign-Pool ALM with kHYPE/stHYPE withdrawal modules, Aave/ERC4626/multi-market lending modules, stepwise fee module. **Toolchain:** Foundry (solc ^0.8.25, cancun, optimizer 10k, no via_ir) + Slither 0.11.3, reusing forge build-info. **Scan:** 246 owned findings — **0 Critical / 3 High / 34 Medium / 57 Low / 152 Info**. **Reports:** `Valantis_STEX_Audit_EN.pdf` (278 pp), `Valantis_STEX_Auditoria_ES.pdf` (274 pp).

> Triage layer. **Nothing here is a bounty submission.** No PoC was submitted anywhere; any disclosure is the user's own action.

Headline

None of the 3 High findings is attacker-exploitable — all verified false positives by reading the real code (and, for the callback, the Valantis-core Sovereign Pool it depends on). This held even on a newer, niche, less-audited AMM: the code is well-structured (ALM-gating, `nonReentrant`, owner timelocks).

High findings — per-site verdict (3/3 FP)

1. arbitrary-send-erc20 → FP · `STEXAMM.onDepositLiquidityCallback` L546

`ERC20(token1).safeTransferFrom(user, msg.sender, _amount1)` with `user = abi.decode(_data)` (arbitrary-looking). **Verified FP via two hard facts:** - `onDepositLiquidityCallback` is `onlyPool` — only

the Sovereign Pool can call it. - The only path that reaches it is `SovereignPool.depositLiquidity`, which in valantis-core (`SovereignPool.sol` L835) is `external onlyALM nonReentrant` — `_onlyALM` reverts unless `msg.sender == alm` (the ALM *is* this STEXAMM). - `STEXAMM.deposit` (L523) calls `depositLiquidity(..., abi.encode(msg.sender))` — the callback `user` is **hard-set to the original depositor**. An attacker cannot inject `user = victim`; they can only move their own approved token1.

This is the #1 real-bug class for Sovereign-Pool AMMs (arbitrary `from` in a deposit callback), and here it is correctly defended. FP.

2. arbitrary-send-eth → FP · `kHYPEWithdrawalModule._update` L1006

`token1.deposit{value: excessNativeBalance}()` wraps native HYPE into WHYPE. The "arbitrary user" is `token1` = the canonical wrapped-native contract (`_getWrappedNativeToken()`), not an attacker address. `_update` is `internal`, reached only from owner-gated / `nonReentrant` flows. FP.

3. reentrancy-eth → FP · `kHYPEWithdrawalModule.settlePendingWithdrawalsWithPoolReserves` L744

`onlyOwner nonReentrant whenPoolNotLocked`. Every external call is to a trusted/immutable dependency — `stakingManager` (immutable), `stex`, `_getWrappedNativeToken()` (WHYPE). The explicit `nonReentrant` guard plus owner-gating plus the pool-lock check close any re-entry. FP.

Medium (34) — class characterization (spot-checked)

Class	Count	Verdict
`unused-return`	13	Noise — returns intentionally ignored.
`uninitialized-local`	11	Noise — locals assigned before use / default is the intended value.
`reentrancy-no-eth`	6	All in `setProposed*` governance setters (two-step owner timelock pattern). Owner-gated, benign. FP.
`incorrect-equality`	4	Accounting equalities in the kHYPE withdrawal/lending modules. **Team action:** confirm the balance-based `==` (e.g. `_update`, `unstakeExcessToken0`) can't be griefed by a force-send that makes the equality un-satisfiable.

Low (57) / Info (152)

Low dominated by `shadowing-local` (17), `missing-zero-check` (9), `calls-loop` (8), `timestamp` (8), benign reentrancy (11). Info is almost entirely `naming-convention` (144). Standard hygiene; none security-relevant on spot-check. The `missing-zero-check` items (9) are the only mild hardening nits worth a glance (constructor/setters without `address(0)` guards).

Bottom line

- **No submittable bug** found by static scan + triage — third mature-enough target in a row (Lombard, Doppler, Valantis STEX) where all raw Highs are FP. The triage layer is the product.
- Honest limits: the callback FP is backed by reading valantis-core's Sovereign Pool; the other two by reading the module + immutability. No PoC was needed (no candidate surfaced).
- Good-faith notes only (not bounty): review the balance-based strict `==` for force-send griefing; add `address(0)` guards where flagged. **User's own action, nothing submitted.**

Appendix — Evidence trail

- Full scan: `Valantis_STEX_Audit_EN.pdf` (278 pp), `Valantis_STEX_Auditoria_ES.pdf` (274 pp); raw: `valantis_security_results.json`.
- Toolchain: HTTPS submodule fetch → `forge build --build-info --skip test/script` (solc ^0.8.25, cancun, no via_ir) → `slither . --foundry-ignore-compile --json`.
- Target pin: `ValantisLabs/valantis-stex` @ `27b5c74`. Dependency verified: `valantis-core/src/pools/SovereignPool.sol` (`onlyALM` gating).